

[Open in app](#) ↗[Sign up](#)[Sign in](#)**Medium**

To make Medium work, we log user data. By using Medium, you agree to our Privacy Policy, including cookie policy.

[Towards AI](#) · [Follow publication](#)

The 15-File Setup That Turned Claude Code Into My Development Team

6 min read · Mar 31, 2026



Himanshu Bhoir

[Follow](#)[Listen](#)[Share](#)

I was explaining my tech stack to Claude Code for the tenth time in a week.

Same project. Same framework. Same conventions. And yet, every new session started like day one of onboarding a contractor who'd never seen our codebase.

“We use callback-driven validation, not prompt-based.”

“Yes, structured logging only — no print statements.”

“The agent endpoints stream via SSE.”

That's when it hit me: I wasn't using an AI development tool. I was babysitting one.

So I blocked out a weekend, read everything I could find about Claude Code's `.claude/` folder system, and built a setup for my AI agent project.

15 files later, my first prompt every session is productive work. Not context-setting. Not permission-granting. Just the actual task.

To make Medium work, we log user data. By using Medium, you agree to our Privacy Policy, including cookie policy.

How I Configured Claude Code to Stop Acting Like a Junior Dev

Here's exactly what I built, why each piece exists, and how you can do the same.

What Most Developers Get Wrong About Claude Code

Most engineers I know use Claude Code the same way they use ChatGPT — type a question, get an answer, repeat. Every conversation is stateless. Every session is a blank slate.

This works fine for one-off questions. But if you're building a real project — something with a framework, conventions, deployment pipelines, and team standards — you're throwing away context every time you close the terminal.

The `.claude/` folder fixes this. It's a persistent configuration layer that lives in your project repo and loads automatically every session. Think of it as your AI assistant's onboarding document — except it actually reads and follows it.

The structure has six parts, and each one solves a different problem.

The Anatomy of a Production `.claude/` Folder

Here's the full tree of what I built:

```

CLAUDE. To make Medium work, we log user data. By using Medium, you agree to
CLAUDE. our Privacy Policy, including cookie policy.
.claude
  settings.json          ← Shared permission rules
  settings.local.json    ← Personal permissions (gitignored)
  rules/                 ← Auto-enforced contextual rules
  commands/             ← Slash commands for common workflows
  agents/               ← Specialized subagent personas
  skills/               ← Auto-triggered workflow checks

```

Let me walk through each layer.

Layer 1: Rules That Enforce Themselves

Rules are markdown files that activate automatically based on file path globs. You don't invoke them. You don't remember them. They just work.

I wrote three:

rules/code-style.md — Scoped to `**/*.py`

```

---
paths:
  - "**/*.py"
---
# Python Code Style
- Follow PEP 8
- Type hints required on all function signatures
- Use the project logger, never print() or stdlib logging
- Pydantic models for all request/response validation
- Async functions for all FastAPI endpoints

```

rules/agent-patterns.md — Scoped to `agents/**` and `modules/agents/**`

This one only fires when I'm working inside agent directories. It enforces Google ADK callback patterns, proper tool docstrings, session state management, and SSE streaming conventions.

rules/security.md — Scoped to all files

To make Medium work, we log user data. By using Medium, you agree to our Privacy Policy, including cookie policy.

No hardcoded secrets. Everything.

The key insight here is that **the same Claude instance behaves differently depending on where you're working**. Edit a Python utility? You get PEP 8 enforcement. Touch an agent file? You get ADK-specific patterns on top of that. It's like having multiple specialized reviewers built into one tool.

Layer 2: Seven Commands That Cover the Entire SDLC

Commands are slash commands you invoke manually — like `/project:git commit` or `/project:test --fix`. I built seven that cover my full development cycle:

Command	Description
git	Conventional commits, MR creation, branch management
fix	Traces an error to root cause, applies minimal fix
explain	Maps request flow through the codebase with file references
refactor	Safe restructuring with impact analysis across all call sites
test	Runs pytest, analyzes failures, optionally auto-fixes
review	Security-first code review with file:line references
deploy	Pre-deployment checklist for target environment

The git command is my favorite. Instead of configuring commit lint or writing commit message hooks, Claude reads the diff, drafts a conventional commit (`feat(agents): add retry logic to callback chain`), and waits for my approval. It's faster than any hook-based setup I've tried.

Get Himanshu Bhoir’s stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

To make Medium work, we log user data. By using Medium, you agree to our Privacy Policy, including cookie policy.

The fix command is the most useful. You paste an error message, and it searches the codebase for relevant code, traces the execution path from route handler through middleware to the agent layer, and suggests a minimal fix. No unrelated refactoring. No style suggestions. Just the fix.

Layer 3: Agents With Assigned Roles and Models

This is the part most people don't know exists. Claude Code lets you define specialized subagents — each with their own system prompt, tool access, and *model*.

I created three:

```
# .claude/agents/explore-codebase.md
---
name: explore-codebase
description: Answers "how does X work?" questions about the codebase
model: haiku
tools: Read, Grep, Glob
---
You are a codebase explorer for a FastAPI + Google ADK agent project.
Trace full request flows. Reference specific files and line numbers.
Keep answers concise with code references.
```

```
# .claude/agents/code-reviewer.md
---
name: code-reviewer
description: Reviews code for bugs, security, and pattern compliance
model: sonnet
tools: Read, Grep, Glob, Bash
---
Flag actual bugs and logic errors, not style nitpicks.
Check for security issues and proper ADK patterns.
Suggest specific fixes with code snippets.
```

The third is a **test-writer on Sonnet** that generates pytest cases following our project's fi

To make Medium work, we log user data. By using Medium, you agree to our Privacy Policy, including cookie policy.

Here's why this matters: **Haiku is cheap and fast**. When I just need to understand how a request flows through the codebase, I don't need Sonnet-level reasoning. By assigning the right model to the right task, I'm optimizing for both cost and quality at the project configuration level.

Layer 4: The Deny Rules Nobody Talks About

Every `.claude/` setup guide I found focused on what to allow. Permit this bash command. Approve that file read. Make Claude faster by pre-authorizing common operations.

That's half the picture. The other half — the more important half — is what you deny.

Here's my `settings.json`:

```
{
  "permissions": {
    "allow": [
      "Bash(pip install *)",
      "Bash(python *)",
      "Bash(git status)",
      "Bash(git diff *)",
      "Bash(pytest *)",
      "Read", "Write", "Edit"
    ],
    "deny": [
      "Bash(rm -rf *)",
      "Bash(git push --force *)",
      "Bash(git reset --hard *)",
      "Read(.env)",
      "Read(.env.*)"
    ]
  }
}
```

Claude cannot read my `env` files. Cannot force push. Cannot `rm -rf` anything.

Cannot have `settings.json` in the repo. To make Medium work, we log user data. By using Medium, you agree to our Privacy Policy, including cookie policy.

These aren't paranoid restrictions. They're guardrails that mean I never have to hesitate before approving an action. I trust my AI assistant more because I told it exactly where the boundaries are.

And because `settings.json` is committed to the repo while `settings.local.json` is gitignored, the team shares the same safety rules while each developer can add personal overrides for their local environment.

Layer 5: Auto-Triggered Skills

Skills fire automatically when matching conditions are met. No slash command needed.

I have one so far — a security review that triggers whenever I modify API endpoints, configuration files, or Dockerfiles. It runs through a checklist: no hardcoded secrets, Pydantic validation on inputs, no secrets in logs, non-root Docker user.

Zero discipline required. The review happens whether I remember to ask for it or not.

I'm planning to add more — an auto-documentation skill when docstrings change, a dependency audit when `requirements.txt` is modified. The pattern is powerful: identify what you always forget to check, and make it automatic.

The Before and After

Before: Every session started with 5–10 minutes of context-setting. Re-explain the stack. Re-approve permissions. Correct Claude when it generated `print()` instead of structured logging. Hope it remembered our commit conventions.

After: Open terminal. Type the task. Claude already knows the framework, follows the code style, uses the right commit format, and can't touch my secrets. First prompt is productive work.

The compound effect is real. After a week, the setup saved me 30+ minutes a day in context-setting documents. To make Medium work, we log user data. By using Medium, you agree to our Privacy Policy, including cookie policy. ng t written in a wiki nobody reads.

Start With This

You don't need 15 files on day one. Start with three:

1. **CLAUDE.md** — Write your stack, conventions, and what Claude should never do. This alone saves you from repeating yourself.
2. **settings.json** — Add deny rules for destructive commands and `.env` reads. Trust is earned.
3. **One rule file** — Pick your most common code style violation and enforce it.

Then add commands and agents as you notice patterns in your workflow. Every time you catch yourself re-explaining something to Claude, that's a file waiting to be written.

The `.claude/` folder isn't just configuration. It's your AI assistant's job description. And the better the job description, the better the work.

I'm building production AI agents and sharing what actually works (and what breaks). If this was useful, follow along — I write about the gap between AI demos and production reality.

Claude Code +

Vibe Coding +

AI Agent +

Artificial Intelligence +

Ai Engineering +



To make Medium work, we log user data. By using Medium, you agree to our Privacy Policy, including cookie policy.

[Follow](#)

Published in Towards AI

144K followers · Last published 3 hours ago

We build Enterprise AI. We teach what we learn. Join 100K+ AI practitioners on Towards AI Academy. Free: 6-day Agentic AI Engineering Email Guide: <https://email-course.towardsai.net/>

[Follow](#)

Written by Himanshu Bhoir

28 followers · 8 following

Building Agentic AI that transforms how engineers, product teams & orgs work | Teaching AI from the inside out

Responses (2)



Write a response

What are your thoughts?



Complexe Infosystem
Apr 1



Really interesting setup—the idea of turning Claude Code into a “team” through context, sub-agents, and workflows makes it feel less like a tool and more like a collaborative system.



1

[Reply](#)



Max W
Apr 7

To make Medium work, we log user data. By using Medium, you agree to our Privacy Policy, including cookie policy.



Many thanks, interesting setup! would you mind sharing the files? I would love to see and learn by comparing them with own.



Reply

To make Medium work, we log user data. By using Medium, you agree to our Privacy Policy, including cookie policy.

To make Medium work, we log user data. By using Medium, you agree to our Privacy Policy, including cookie policy.